

SAMPLE — illustrative report format • not a real client • shows structure & rigor only

Exposed third-party API credential in public front-end bundle

A live API key was shipped inside a publicly served JavaScript file, usable by anyone who views the site's source.

FINDING ID	CS-SAMPLE-001	SEVERITY	HIGH (calibrated — see Impact)
ASSET	app.<redacted>.com — /assets/main.4f9c1.js	METHOD	Passive discovery + minimal verification
STATUS	Confirmed live at time of scan	CLASS	Exposed secret / credential

WHAT WE FOUND

During passive review of the public front end, the production JavaScript bundle contained a hard-coded API key for a third-party email/messaging provider. The key is served to every visitor and readable directly from the page source — no authentication, no obfuscation.

```
// /assets/main.4f9c1.js (public, minified)
const EMAIL_API_KEY = "SG.xxxxxxxx<redacted>xxxxxxx";
fetch(PROVIDER_URL, { headers: { Authorization: `Bearer ${EMAIL_API_KEY}` } })
```

HOW WE VERIFIED IT (VERIFICATION-FIRST)

We confirmed the key was **live and mapped to your account** using a **single, minimum-necessary metadata call** to the provider's identity endpoint. **No email was sent. No data was read, modified, or exfiltrated.** Verification stopped the moment validity was established — that limit is written into our Rules of Engagement.

REAL-WORLD IMPACT — CALIBRATED HONESTLY

What an attacker could do

- Send email as your brand (phishing, reputation damage)
- Exhaust your sending quota / rack up provider cost
- Read delivery metadata exposed by that key's scope

What it does NOT allow

- No access to your customer database or app back end
- No access to payment or auth systems
- Blast radius is brand + provider account, not full breach

Why High, not Critical: the key is trivially discoverable and immediately abusable (raises it), but its scope is limited to one third-party service and cannot pivot into your core systems (caps it). We rate what an attacker can actually do — not how alarming the screenshot looks.

REMEDIATION

- **Rotate the key now** in the provider dashboard — the exposed value must be treated as compromised.
- **Move the secret server-side.** Front-end code should call your own backend, which holds the key and proxies the request — the browser never sees it.
- **Scope the new key down** to the minimum permissions the feature needs.
- **Add secret scanning to CI** so a key can't reach a public bundle again.